

Entrega — Uso de Inteligência Artificial

Aluno: Gabriel Trajano. Documento para o AVA: link do repositório público e registro dos prompts utilizados com IA, com respostas correspondentes (resumidas).

Link do repositório (GitHub) \ <https://github.com/GabrielTrajano078/MeuBot>

Interação 1 — visão geral do sistema

Prompt / contexto:

É um trabalho de faculdade: um bot do Discord em Elixir com o Nostrum, Mix, Tesla (ou HTTPoison) e Jason. O professor quer pelo menos sete comandos funcionais, cada um batendo numa API REST diferente, um deles encadeando duas APIs, outro com persistência em arquivo JSON entre sessões, e tudo organizado em módulos com nomes claros: aplicação/supervisor, consumer que despacha por pattern matching, comandos HTTP e um módulo só para ler/gravar o JSON. Também pediram OTP com GenServer para estado compartilhado, sem variável global. Por onde você começaria e quais pastas ou módulos fariam sentido sem amarrar tudo num arquivo só?

Resposta da IA (resumo):

Resposta da IA: sugeriu um projeto Mix com `--sup`, dependências Nostrum, Tesla, Hackney e Jason; `MeuBot` como ponto de entrada do Application com supervisor; `MeuBot.Consumer` só para eventos do Discord e roteamento de mensagens; `MeuBot.Commands` com uma função pública por comando e helpers privados usando pipe; `MeuBot.Store` isolando leitura/escrita do JSON; `MeuBot.Reminders` como GenServer que carrega do Store na subida e grava a cada inclusão; token só por variável de ambiente; README e `.gitignore` incluindo o arquivo de dados local.

Interação 2 — eventos do Discord e “loop” de mensagens

Prompt / contexto:

No CLI a gente pensa em recursão de cauda com `IO.gets`. No bot, o fluxo é reativo: chega `MESSAGE_CREATE`, eu preciso ignorar bot, fazer trim e decidir qual comando rodar. Como desenhar isso para ficar elegante de explicar na banca, sem virar um único `cond` gigante misturado com chamada HTTP?

Resposta da IA (resumo):

Resposta da IA: recomendou manter o `Consumer` fino: uma função `dispatch/2` (ou equivalente) com várias cláusulas fazendo pattern matching no texto já normalizado (`!ping`, `!clima " <> cidade`, etc.); respostas enviadas por `Nostrum.Api.Message.create/2`; lógica pesada e HTTP ficam em `Commands`; casos incompletos (`!clima` sem cidade) em cláusulas separadas com mensagem de uso; `catch-all` no fim para ignorar o que não for comando.

Interação 3 — comandos com um parâmetro e com vários argumentos

Prompt / contexto:

Alguns comandos são um argumento só (!clima Fortaleza), outros precisam de três tokens fixos (!conv 100 USD BRL) e o !wiki aceita várias palavras até o fim da linha. Em Elixir, como costuma-se combinar pattern matching na string com `String.split` controlado para não virar um bloco só de `if` aninhado, e ainda separar “parse da mensagem” de “chamar a API”?

Resposta da IA (resumo):

Resposta da IA: usar concatenação de binários no consumer só onde bater o formato (!conv " <> resto e depois `String.split` em partes); para !wiki guardar o resto da linha como uma única string de busca; validar contagem de argumentos no consumer ou devolver erro tipado de `Commands`; manter URLs e `URI.encode_query` concentrados no módulo de comandos.

Interação 4 — estado imutável, GenServer e I/O de rede

Prompt / contexto:

Quero manter o estilo funcional: nada de estado global mutável. A lista de lembretes precisa ser compartilhada entre mensagens e ainda persistir em disco. Inserir na lista e listar deveriam passar por um processo único. Tem algum pitfall comum quando mistura GenServer com chamadas HTTP lentas no mesmo processo que atende o Discord?

Resposta da IA (resumo):

Resposta da IA: colocou o GenServer só para fila de lembretes e arquivo; as requisições Tesla rodam no processo da Task que o Nostrum já dispara por evento (ou no fluxo do handler), para não bloquear o GenServer com rede longa; o GenServer mantém a lista em memória imutável e só chama `Store.write_notes/1` após cada add; leitura inicial no `init/1` a partir do JSON.

Interação 5 — arquivo JSON e sessões

Prompt / contexto:

O arquivo tem que sobreviver reinício do bot. Na primeira execução pode nem existir. Quando existe, vem texto cru do disco. Como isolar essa parte para o resto trabalhar com listas de strings em Elixir e só serializar de volta quando realmente mudar um lembrete?

Resposta da IA (resumo):

Resposta da IA: `Store.read_notes/0` devolve lista vazia se faltar arquivo, estiver vazio ou JSON inválido; `write_notes/1` grava um objeto `{"notes": [...]}` com Jason; diretório criado com `File.mkdir_p!/1` se preciso; o `GenServer` é quem orquestra “memória + persistência” para não espalhar `File.write` pelo consumer.

Interação 6 — erros, HTTP e mensagens no canal

Prompt / contexto:

API externa cai, CEP inválido, cidade que o geocode não acha, formato estranho de JSON da Wikipedia. Não precisa tratamento enterprise, mas quero mensagens curtas no Discord e não explodir o processo inteiro. Onde colocar cada tipo de checagem para não duplicar lógica entre comandos?

Resposta da IA (resumo):

Resposta da IA: padronizar retorno `{:ok, texto}` ou `{:error, motivo}` em `Commands`; funções privadas `map_*_response` por API; o consumer só repassa para `Message.create`; Tesla com timeout de recepção no Hackney; encadeamento `!curiosidade` com `with` parando no primeiro passo que falhar.

Documento gerado a partir do registro de uso de IA do projeto MeuBot (Discord / Elixir / Nostrum), no mesmo formato lógico do entregável AgendaCLI.